

Tweets Emotion Classification using Word Embeddings and CNN - Report

1. Introduction

Emotion classification in text is a fundamental task in Natural Language Processing (NLP), with applications in sentiment analysis, customer feedback evaluation, and social media monitoring. In this project, we aim to classify tweets into six emotion categories: **sadness, joy, love, anger, fear, and surprise**.

To achieve this, we implemented two Convolutional Neural Network (CNN) models using different word embedding techniques:

- Pretrained **GloVe embeddings**
- Custom-trained **Word2Vec embeddings**

The goal is to compare their performance and analyze the effect of different sequence padding lengths on model accuracy.

2. Data Preprocessing

The dataset consists of over 400,000 tweets labeled with one of six emotions. Initial data inspection showed that:

- The dataset is slightly imbalanced, with *joy* being the most frequent class and *surprise* the least.
- Tweets are generally short, with an average length of around 19 words.

Preprocessing Steps:

- Removed URLs using regular expressions
- Normalized whitespace (removed extra spaces)
- No further cleaning was applied since the dataset was already well-preprocessed (no mentions, hashtags, or punctuation)

This minimal preprocessing approach preserved semantic meaning while ensuring clean input for tokenization.

3. Tokenization and Padding

We used the Keras Tokenizer to convert text into sequences of integers.

- Vocabulary size was limited to **100,000 words**

- Each word was mapped to a unique index
- Out-of-vocabulary words were handled using a special token (<OOV>)

Sequences were padded using **post-padding** to ensure equal-length inputs for the CNN.

4. Word Embeddings

4.1 GloVe Embeddings

We used pretrained **GloVe Twitter embeddings (50-dimensional)**. An embedding matrix was constructed where:

- Each row corresponds to a word index
- Each value is the corresponding GloVe vector
- Words not found in GloVe were assigned zero vectors

The embedding layer was set to **non-trainable** to preserve pretrained knowledge.

4.2 Word2Vec Embeddings

We trained a Word2Vec model using the dataset itself:

- Vector size: 50
- Window size: 5
- Minimum word frequency: 2

An embedding matrix was built similarly to GloVe, but using:

```
w2v_model.wv[word]
```

Unlike GloVe, the embedding layer was set to **trainable**, allowing the model to fine-tune embeddings during training.

5. Model Architecture

Both models used the same CNN architecture to ensure a fair comparison:

- Embedding Layer

- Conv1D (128 filters, kernel size = 5, ReLU)
- MaxPooling1D
- Conv1D (128 filters, kernel size = 3, ReLU)
- Flatten
- Dense (64 units, ReLU)
- Dropout (0.5)
- Output Layer (6 units, Softmax)

This architecture allows the model to capture both local and global patterns in text.

6. Training Setup

- Loss Function: Categorical Cross-Entropy
- Optimizer: Adam
- Batch Size: 128
- Epochs: 5
- Validation Split: 10%

Stratified splitting was used to preserve class distribution in training and testing sets.

7. Results and Comparison

7.1 Model Performance

Model	Embedding Type	Trainable	Validation Accuracy	Test Accuracy
CNN + GloVe	Pretrained	No	92.19%	91.38%
CNN + Word2Vec	Custom-trained	Yes	93.69%	93.23%

Analysis:

- The Word2Vec model outperformed GloVe by approximately **1.85%**
- This improvement is due to Word2Vec capturing **dataset-specific semantic relationships**
- Both models showed minimal overfitting, with validation and test accuracies closely aligned

7.2 Effect of Train-Test Split Ratio

To evaluate the impact of different data splitting strategies, we experimented with two train-test split ratios: 80/20 and 70/30.

The results show that both models achieved slightly higher accuracy when using the 80/20 split compared to the 70/30 split. For the GloVe-based model, accuracy decreased from 91.38% to 90.96%, while for the Word2Vec-based model, accuracy decreased from 93.23% to 93.08%.

This behavior is expected, as the 80/20 split provides a larger training set, allowing the model to learn more representative patterns. In contrast, the 70/30 split reduces the amount of training data, which can slightly affect the model's performance.

However, the difference in accuracy between the two splits is relatively small, indicating that both models are stable and generalize well even with less training data. Overall, the 80/20 split offers a better balance between training and evaluation.

8. Impact of Padding Length on Model Performance

We evaluated different sequence lengths to analyze their impact on performance.

Padding Length	Validation Accuracy
50	91.96
100	92.00
200	92.14
500	92.19
700	92.05
1000	92.15

Analysis:

- Performance remains stable across different lengths (~92%)
- Slight improvement up to length 500
- No significant gain beyond 200 tokens

Interpretation:

- Tweets are short (average ≈ 19 words)
- Most useful information is contained in the first few tokens
- Larger padding introduces unnecessary zeros, increasing computational cost without improving performance

Conclusion:

A padding length of **200–500** provides the best balance between performance and efficiency.

9. User Input Prediction

A prediction pipeline was implemented to classify new user input:

- Input text is cleaned
- Tokenized using the same tokenizer
- Padded to a fixed length
- Passed to the trained Word2Vec model
- Output mapped to emotion label

This allows real-time emotion classification for new tweets.

10. Conclusion

In this project, we successfully built and compared two CNN-based emotion classification models using different embedding techniques.

Key findings:

- Word2Vec embeddings outperform GloVe due to domain-specific learning
- CNN is effective for capturing local patterns in text
- Padding length has minimal impact beyond a certain threshold
- Proper preprocessing and embedding selection significantly influence performance

Overall, the best-performing model (Word2Vec + CNN) achieved a test accuracy of **93.23%**, demonstrating strong generalization and effectiveness.